Project: Naive Bayes Classifier

Course: CS230/561

October 2026 (Version 1)

1 Objective

The goal of this project is to implement a **binary Naive Bayes classifier from scratch** for SMS text classification. You will apply the classifier to the **SMS Spam Collection** (binary classification: spam vs ham) dataset, which is available on the UCI Machine Learning Repository:

`https://archive.ics.uci.edu/dataset/228/sms+spam+collection`

2 Bayes Classifier Framework

We want to predict the class C of a document $\boldsymbol{w}_d = (w_{d,1}, w_{d,2}, \ldots, w_{d,n_d})$, where n_d is the number of words in document d and $w_{d,i}$ is a word instance in document d.

2.1 Bayes' Rule

$$P(C \mid \boldsymbol{w}_d) = \frac{P(\boldsymbol{w}_d \mid C) \cdot P(C)}{P(\boldsymbol{w}_d)}.$$

- $P(C)$: **Prior probability** of class C, which is computed as the fraction of training documents in class C.
$$P(C) = \frac{\#\text{ documents in class } C}{\text{total } \#\text{ documents}}$$
- $P(\boldsymbol{w}_d \mid C)$: **Likelihood**. Probability of observing the words in document d given class C.
- $P(\boldsymbol{w}_d)$: **Evidence**. Same for all classes, can be ignored in practice.

2.2 Naive Assumption

Computing $P(\boldsymbol{w}_d \mid C)$ directly can be difficult, so we use the Naive Bayes assumption. We make the *Naive Bayes assumption*: word-presence indicators are conditionally independent given the class.

$$P(\boldsymbol{w}_d \mid C) \approx \prod_{i=1}^{n_d} P(w_{d,i} \mid C)$$

2.3 Estimating Likelihoods

In order to compute $P(\boldsymbol{w}_d \mid C)$, we need to estimate the probability of each word given a class. For this project, we will use a binary (presence-based) likelihood model.

2.4 Binary ON/OFF (Presence-Based) Likelihood

This model only considers whether a word is **present or absent**, not how many times it occurs. It is useful for short messages (like SMS), where the presence of words like "free" or "prize" is more important than frequency.

For each word w:

$$P(w \mid C) = \frac{\#\text{ documents in class } C \text{ containing } w + \alpha}{\#\text{ documents in class } C + 2\alpha}.$$

- Here we treat w as a binary ON/OFF feature: does it appear at least once in the document?
- If w does not appear in a document, we use $1 - P(w \mid C)$ in the product.
- Example: In spam detection, the presence of the word "win" is already a strong signal; repetition adds little.

Thus, for an SMS message d with binary indicators $\mathbb{I}_d(w) \in \{0, 1\}$ for each vocabulary word w, the likelihood is

$$P(\boldsymbol{w}_d \mid C) \approx \prod_{w \in V} \left[P(w \mid C)^{\mathbb{I}_d(w)} \cdot (1 - P(w \mid C))^{1-\mathbb{I}_d(w)}\right].$$

2.5 Prediction Rule

Once we have estimated the prior probabilities $P(C)$ and the presence-based likelihoods for all vocabulary words and classes, we can classify a new SMS message.

2.5.1 Computing a Score for Each Class

For each possible class C (`spam` or `ham`), we compute a log-score:
For Binary:

$$\text{score}(C, \boldsymbol{w}_d) = \log P(C) + \sum_{w \in V} \Big[\mathbb{I}_d(w) \log P(w \mid C) + (1 - \mathbb{I}_d(w)) \log(1 - P(w \mid C))\Big].$$

- The first term $\log P(C)$ is the **prior**, which reflects how common the class is in the training data.
- The second term adds up the contributions of all words in the document.
- We use **logarithms** so that adding small numbers is easier and more stable than multiplying many tiny probabilities.

2.5.2 Choosing the Class

The predicted class is simply the one with the largest score:

$$\hat{C}(d) = \arg\max_{C} \ \text{score}(C, \boldsymbol{w}_d).$$

This means: "Choose the class with the largest log-score for the given SMS message."

3 Evaluating the Classifier

Once we can predict labels for test messages, we must evaluate how well the classifier works.

3.1 Accuracy.

Accuracy is the fraction of test documents that are classified correctly:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of test documents}}.$$

3.2 Confusion Matrix.

A confusion matrix shows how often documents of one true class were predicted as another class.

- Rows = actual classes
- Columns = predicted classes
- The diagonal entries show correct predictions.

For example, in a binary spam/ham task:

	Predicted: ham	**Predicted: spam**
Actual: ham	480	20
Actual: spam	15	85

Here:

- 480 ham messages correctly identified as ham.
- 85 spam messages correctly identified as spam.
- 20 ham messages mistakenly flagged as spam (false positives).
- 15 spam messages missed (false negatives).

Other Metrics (optional). From the confusion matrix, we can also define:

- **Precision** (for spam): fraction of predicted spam that was truly spam.
- **Recall** (for spam): fraction of actual spam that was correctly detected.

These are helpful when classes are imbalanced.

4 Assignment Tasks

1. Download and preprocess the dataset:
    - Lowercase text
    - Remove punctuation
    - Tokenize into words
2. Build a vocabulary from training data.
3. Implement functions to compute:
    (a) Prior probabilities $P(C)$
    (b) Likelihoods $P(\boldsymbol{w}_d \mid C)$
    (c) Posterior scores $\log P(C \mid \boldsymbol{w}_d)$
4. Train the classifier on the training set.
5. Test on the test set and compute accuracy.

5 Submission guidelines

Submit a single **R script (**`.R`**)** containing your code. Write your explanations, results, and summary as **R comments** (lines starting with `#`).

- Name the file `<teamname>_nb.R`. The script must run from top to bottom without errors, `source("<teamname`
with `SMSSpamCollection` in the same folder.
- Organize it with clear section headers written as comments, for example `# ===== 1. Data Loading and Preprocessing =====`. It should include:
- **Data Loading and Preprocessing:** Code, plus a short comment explaining how the SMS messages were cleaned, tokenized, and split into training and test sets.
- **Binary Naive Bayes Implementation (from scratch):** Functions to compute class priors, presence-based likelihoods, and log-scores.
- **Evaluation:** Print the following (comments should state what each output shows):
  - Accuracy on the test data
  - Confusion matrix for spam/ham classification
  - Predictions for the example SMS messages below

Message	**Expected Class**
"Win free tickets now!!!"	spam
"Are you coming to the meeting?"	ham
"URGENT! You won $1000"	spam
"See you tomorrow at lunch"	ham

- **Analysis and summary (as comments):** List and discuss the top 10 words most indicative of `spam` and the top 10 words most indicative of `ham`. Add a brief summary (about 4–6 sentences) of what is learned from the project, as a comment block at the end of the script.

Grading Rubric

Category	**Criteria**	**Points**
1. Data Preprocessing	Text Cleaning & Tokenization	10
	Train/Test Split	5
2. Binary Naive Bayes Implementation	Prior Computation	5
	Likelihood Estimation	10
	Log Posterior / Scoring	10
3. Evaluation & Results	Accuracy Calculation	5
	Confusion Matrix (spam/ham)	5
	SMS Test Cases	5
4. Analysis and summary		5
Total		**60**